

Create a new repository

Import repository

Workspace

tsam061106

Project*

SE

Repository name*

Private projects can only contain private repositories

tsam061106-Museum-Ticketing-and-Artifact-Guide-System

Access level

☒ Private repository

Uncheck to make this repository public. Public repositories typically contain open-source code and can be viewed by anyone.

Include a README?

Yes, with a template

Default branch name

e.g., 'main'

Include .gitignore?

Yes (recommended)

> Advanced settings

Cancel

Create repository

Fig1 : Creation Of Repository

Bitbucket

For you

Recent

Starred

Pull requests

Repositories

Recent

Museum-Ticketing-and-Artifa...

Source

Commits

Branches

Pull requests

Merge queues

Packages

Pipelines

Tests

Deployments

Jira work items

Security

Downloads

View all repositories

Projects

Packages

Snippets

Search

Create

tsam061106 / SE

Museum-Ticketing-and-Artifact-Guide-System

Pull requests

Invite

Clone

...

Here's where you'll find this repository's source files. To give your users an idea of what they'll find here, [add a description to your repository](#).

main

Files

Filter files

Name	Size	Last commit	Message
DFD-Diagrams		2026-08-29	docs(df): added DFD Level 1 and Level 2
Documentation		3 days ago	docs(srs): updated administrator requirement
Project-Report		2 days ago	docs: add comprehensive project report
UML-Diagrams		2026-08-27	docs(uml): added Class Diagram
src		2 days ago	refactor: refactor code and add system test si
tests		2 days ago	refactor: refactor code and add system test si
.gitignore	255 B	2026-08-20	chore: initial project setup and directory struc
README.md	11.13 KB	22 hours ago	docs: updated README
package.json	551 B	17 hours ago	chore: final documentation cleanup

Fig2: Repository Homepage

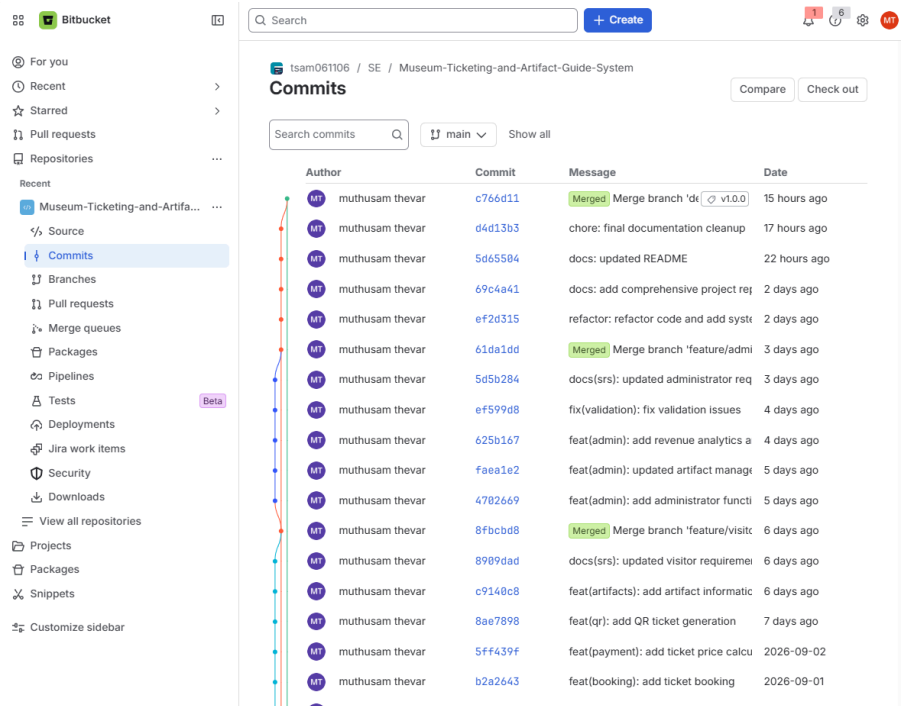


Fig3: Commit History

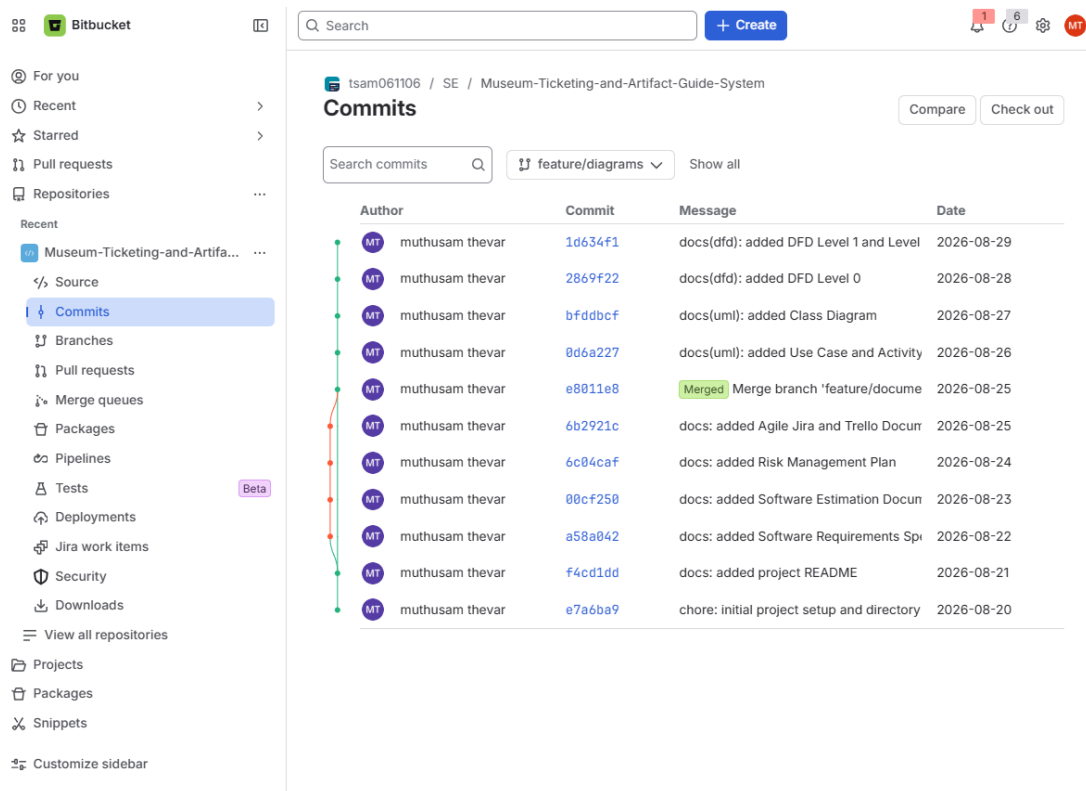


Fig4: Individual Commit Diff

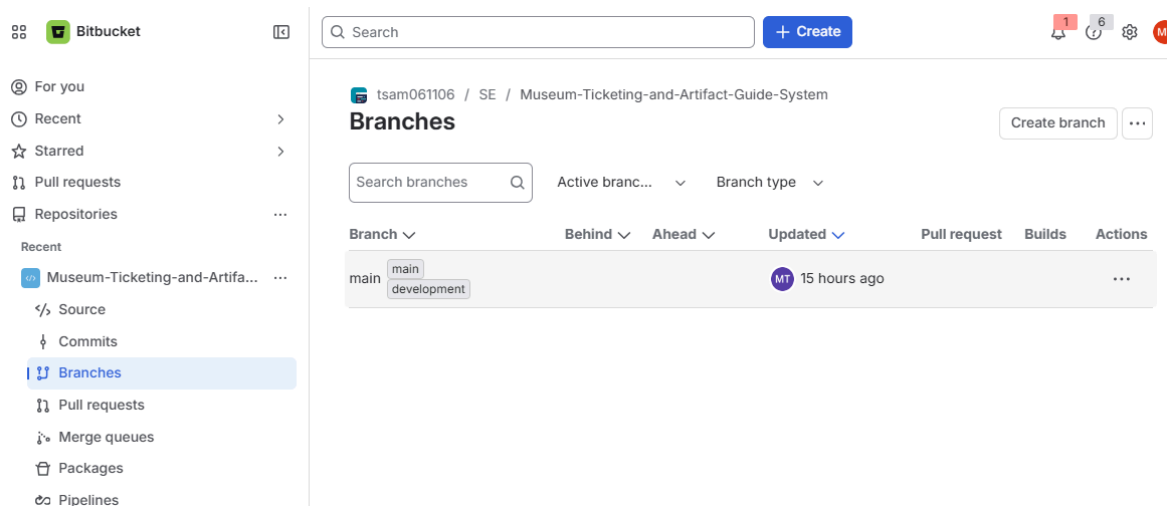


Fig5: Branch & Collaboration Network Graph

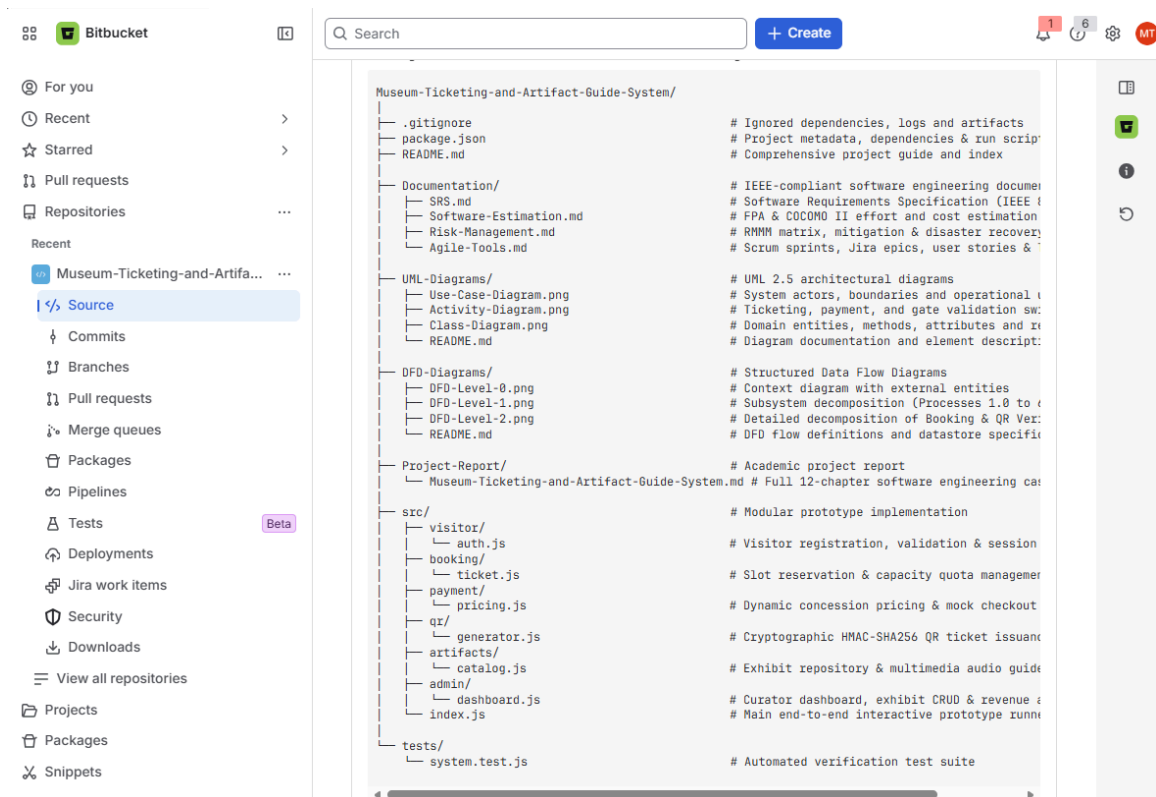


Fig 6: File Version History (Incremental Evolution)

Aim : To develop different versions for the selected case study using any Software Configuration Management (SCM) tool.

Theory :

1. Introduction

Software Configuration Management (SCM) is a systematic process used to **identify, organize, control, and track changes** made during software development. As a software project develops, different files such as source code, requirements, designs, reports, and configuration files are modified many times. SCM helps maintain these changes in an organized manner.

SCM allows developers to maintain **different versions of a software project** and provides a record of how the project has changed over time. If a new change causes an error, an earlier stable version can be identified and restored. It also helps multiple developers work on the same project while reducing conflicts and accidental loss of work.

The major activities involved in SCM include:

- Version control
- Change management
- Configuration identification
- Revision history maintenance
- Branching and merging
- Access and collaboration management
- Backup and recovery of previous versions

2. SCM Tools

Different SCM tools are available for managing software versions, tracking changes, and supporting collaboration. Some commonly used tools are GitHub, Apache Subversion, Mercurial, Perforce Helix Core, Apache Maven, IBM Rational ClearCase, and Plastic SCM.

3.1 GitHub

GitHub is a web-based platform used for software development, repository hosting, version management, and collaboration. It allows developers to store project files and maintain their development history.

GitHub provides features such as repositories, branches, pull requests, issues, collaboration, and file history. Developers can view previous changes, compare versions, and work on different features independently.

Important features include:

- Repository management
- Version history
- Branches
- Pull requests
- Issue tracking
- Collaboration
- File change history

GitHub is widely used for software development and collaborative projects.

3.2 Apache Subversion (SVN)

Apache Subversion, commonly called SVN, is a centralized Software Configuration Management and version-control system. It maintains project files in a central repository and records changes as revisions.

Developers can update their local files, make modifications, and commit those changes to the central repository. SVN also supports branching, tagging, access control, and revision history.

Important features include:

- Central repository
- Revision history
- Commit and update operations
- Branching and tagging
- Change tracking
- Access control
- Conflict management

SVN is useful for organizations that prefer a centralized approach to version control.

3.3 Mercurial

Mercurial is a distributed version-control system used to manage source code and track changes in software projects. It allows developers to maintain project history locally and synchronize changes with other repositories.

It supports commits, branches, merging, and revision history. Mercurial is designed to provide efficient version management while keeping the workflow relatively simple.

3.4 Perforce Helix Core

Perforce Helix Core is an SCM and version-control platform designed for managing large software projects and large files. It provides version tracking, branching, merging, access control, and collaboration features.

It is particularly useful when projects contain large codebases, binary files, multimedia assets, or other large project resources.

3.5 Apache Maven

Apache Maven is primarily a build and project management tool, especially for Java projects. It helps manage project dependencies, compilation, testing, packaging, and builds.

Maven uses a project configuration file to define dependencies and build information. It helps developers maintain a consistent project structure and build process.

3.6 IBM Rational ClearCase

IBM Rational ClearCase is an enterprise-level Software Configuration Management tool used to manage different versions and configurations of software. It provides version control, branching, merging, access control, and configuration management features.

It is mainly suitable for large organizations and complex software development environments where strict configuration control is required.

3.7 Plastic SCM

Plastic SCM is a version-control and SCM tool designed for software and game development. It provides features such as branching, merging, version history, distributed development, and collaboration.

It can handle both source code and large binary files, making it useful for projects where different types of project assets need to be managed.

4. Comparison of SCM Tools

SCM Tool	Type / Approach	Main Purpose	Suitable For
GitHub	Distributed / Cloud-based platform	Version control and collaboration	Software and team projects
Apache SVN	Centralized	Centralized version control	Organizations using a central repository
Mercurial	Distributed	Source-code version control	Software development
Perforce Helix Core	Centralized / Distributed	Code and large-file management	Large-scale projects
Apache Maven	Build and project management	Dependency and build management	Java projects
IBM Rational ClearCase	Enterprise SCM	Configuration and version management	Large enterprise projects
Plastic SCM	Distributed / Centralized	Version control and collaboration	Software and game development

The main difference between these tools is their architecture and purpose. SVN and ClearCase are commonly associated with centralized configuration management, while Mercurial supports distributed version control. GitHub provides an online platform for repository hosting and collaboration. Perforce Helix Core and Plastic SCM are useful for projects involving large files and complex development. Maven is different from traditional source-code SCM tools because its primary focus is build and dependency management.

5. Advantages of Software Configuration Management

SCM provides several benefits throughout the software development life cycle. It helps maintain the integrity of project files and makes the development process more organized.

The major advantages are:

- **Version Management:** Different versions of the project can be maintained.
- **Change Tracking:** Every important modification can be recorded and monitored.
- **Collaboration:** Multiple developers can work on the same project.
- **Recovery:** Previous stable versions can be restored when required.
- **History:** Developers can identify what changes were made and when.
- **Consistency:** Helps maintain consistent project files.
- **Reduced Risk:** Prevents accidental loss or overwriting of important work.

Conclusion

Software Configuration Management is an important part of software development because it provides systematic control over project changes and versions. GitHub provides online project hosting and collaboration features, while Apache Subversion (SVN) provides centralized version control and revision management. These tools help developers maintain different versions, track changes, and manage software projects in an organized manner.

LOs achieved: LO3, LO5, LO6

POs achieved: PO1, PO2, PO3, PO4, PO5, PO8, PO9, PO10, PO11, PO12

PSOs achieved: PSO1, PSO2